

🕒 Cập nhật tháng 8 năm 2024

[Bài đọc] Login/Register bằng REST API (POST JSON)

1. Register bằng REST API

- Tạo lớp User với các trường dữ liệu id, username, password, email:

```
@Entity 9 usages
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String username;
    private String password;
    private String email;
}
```

- Sử dụng Spring Data JPA để thao tác với CSDL;

```
@Repository no usages
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username); no usages
    boolean existsByUsername(String username); no usages
    boolean existsByEmail(String email); no usages
}
```

- Tạo lớp AuthService để xây dựng các phương thức login/register:

```

@Service no usages
public class AuthService {
    @Autowired
    private UserRepository userRepository;
    @Autowired
    private PasswordEncoder passwordEncoder;

    public User register(User user) { no usages
        if (userRepository.existsByUsername(user.getUsername()) ||
            userRepository.existsByEmail(user.getEmail())) {
            throw new RuntimeException("Username or email already exists");
        }
        user.setPassword(passwordEncoder.encode(user.getPassword()));
        return userRepository.save(user);
    }
}

```

- Tạo Controller để kiểm thử bằng Postman:

```

@RestController
@RequestMapping("/api/auth")
public class AuthController {
    @Autowired
    private AuthService authService;

    @PostMapping("/register")
    public ResponseEntity<Map<String, Object>> register(@RequestBody User user) {
        Map<String, Object> response = new HashMap<>();
        try {
            User savedUser = authService.register(user);
            response.put("message", "User registered successfully");
            response.put("userId", savedUser.getId());
            return ResponseEntity.status(201).body(response);
        } catch (RuntimeException e) {
            response.put("error", e.getMessage());
            return ResponseEntity.badRequest().body(response);
        }
    }
}

```

- Cấu hình Security cho phép gọi các API liên quan đến Auth mà không cần phải xác thực:

```

@Configuration
@EnableWebSecurity

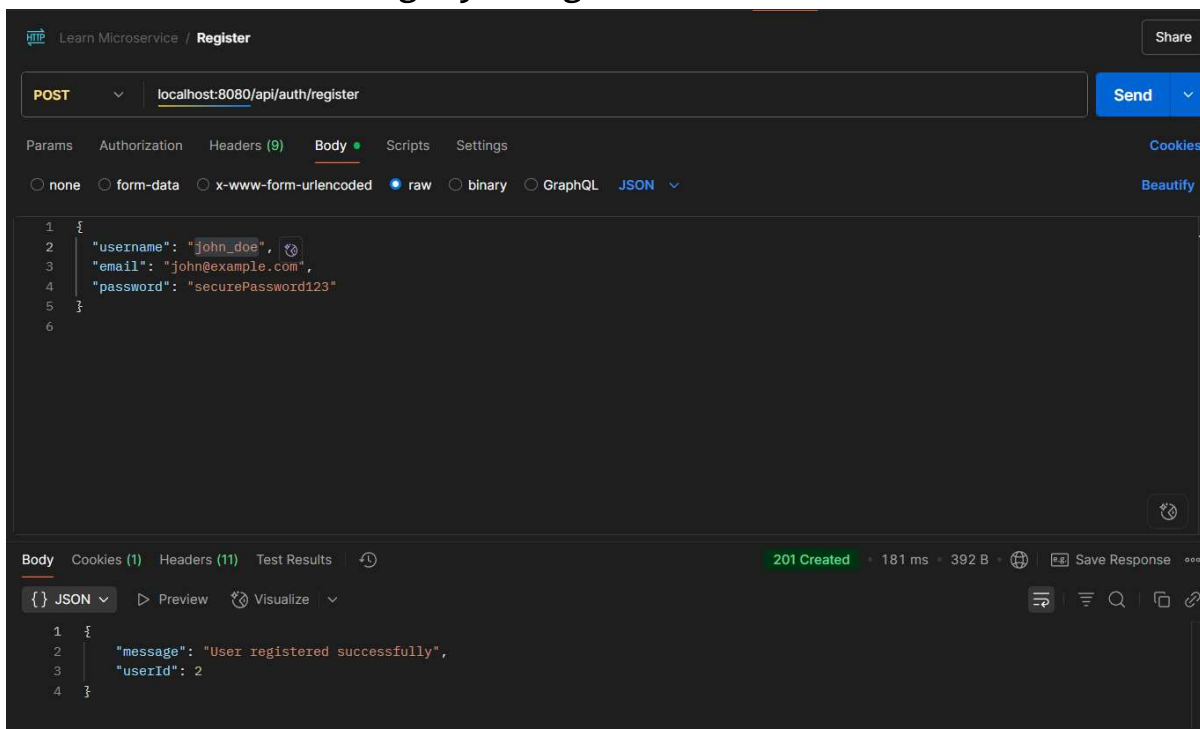
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .anyRequest().authenticated()
            );
        return http.build();
    }
}

```

- Kiểm thử đăng ký bằng Postman:



2. Login bằng REST API

- Xây dựng phương thức login trong AuthService để xác thực người dùng theo username và password:

```

public User login(String username, String password) {
    User user = userRepository.findByUsername(username)
        .orElseThrow(() -> new RuntimeException("Invalid username or password"));
    if (!passwordEncoder.matches(password, user.getPassword())) {
        throw new RuntimeException("Invalid username or password");
    }
    return user;
}

```

- Tạo API trong AuthController để kiểm thử với Postman:

```

@PostMapping("/login")
public ResponseEntity<Map<String, Object>> login(@RequestBody Map<String, String> credentials) {
    Map<String, Object> response = new HashMap<>();
    try {
        User user = authService.login(
            credentials.get("username"),
            credentials.get("password")
        );
        response.put("message", "Login successful");
        response.put("username", user.getUsername());
        return ResponseEntity.ok(response);
    } catch (RuntimeException e) {
        response.put("error", e.getMessage());
        return ResponseEntity.status(401).body(response);
    }
}

```

- Kiểm thử API với Postman:

